МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»

Навчально-науковий фізико-технічний інститут

Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Аналіз кіберінцидентів»

Практичне заняття № 1 — Логістична регресія в системах машинного навчання

Виконав: Вовкотруб Олександр Віталійович
Група: ФБ-61мн

Перевірив: Наконечна Юлія Володимирівна

Київ — 2026

Зміст

1 Мета роботи **2**

2 Постановка задачі **2**

3 Хід виконання **2**

3.1 Підготовка даних 2

3.2 Базова лінія 3

3.3 Логістична регресія 3

3.4 Відбір ознак і виявлена пастка 4

3.5 Оцінка складності задачі 4

3.6 Інтерпретація коефіцієнтів 5

3.7 Поріг відсікання та логістична функція 6

4 Результати **7**

5 Аналіз **8**

6 Висновки **9**

7 Джерела **9**

1 Мета роботи

Побудувати класифікатор виконуваних файлів Windows на шкідливі та безпечні на основі моделі логістичної регресії, оцінити його якість коректними для незбалансованих даних метриками та дослідити, скільки ознак насправді потрібно для розв’язання задачі.

2 Постановка задачі

Дано датасет UCI Malware Executable Detection: 373 виконуваних файли, для кожного 531 бінарна ознака `F_1` … `F_531`, утворена з гібридних характеристик — шістнадцяткових послідовностей байтів та викликів DLL. Мітка `Label` набуває значень `malicious` (301 файл) та `non-malicious` (72 файли).

Відповідно до методичних вказівок, позитивною подією вважаємо клас **«шкідливий»** (за аналогією з прикладом медичної діагностики, де позитивний результат — «Хворий»).

Дані мають дві особливості, що визначили хід роботи:

- **дисбаланс класів** 301 : 72, тобто 4.2 : 1;
- **p > n** — ознак (531) більше, ніж прикладів (373), що прямо суперечить вимозі методички: *«кількість прикладів має в кілька разів перевищувати кількість вхідних ознак»*.

3 Хід виконання

3.1 Підготовка даних

Пропущених значень немає. Усі 531 ознака бінарні (`{0, 1}`), розрідженість — 80.3 % нулів. Стандартизація не застосовувалась: усі ознаки вже в одній шкалі, а `StandardScaler` потрібен для різномасштабних числових ознак.

Виявлено **28 константних ознак** (тотожно нульових) — вони не здатні нічого розрізнити й були видалені. Залишилось 503 ознаки.

Вибірку поділено у співвідношенні 75 : 25 зі **стратифікацією** — без неї при такому дисбалансі тестова вибірка могла б отримати непредставницьку частку безпечних файлів.

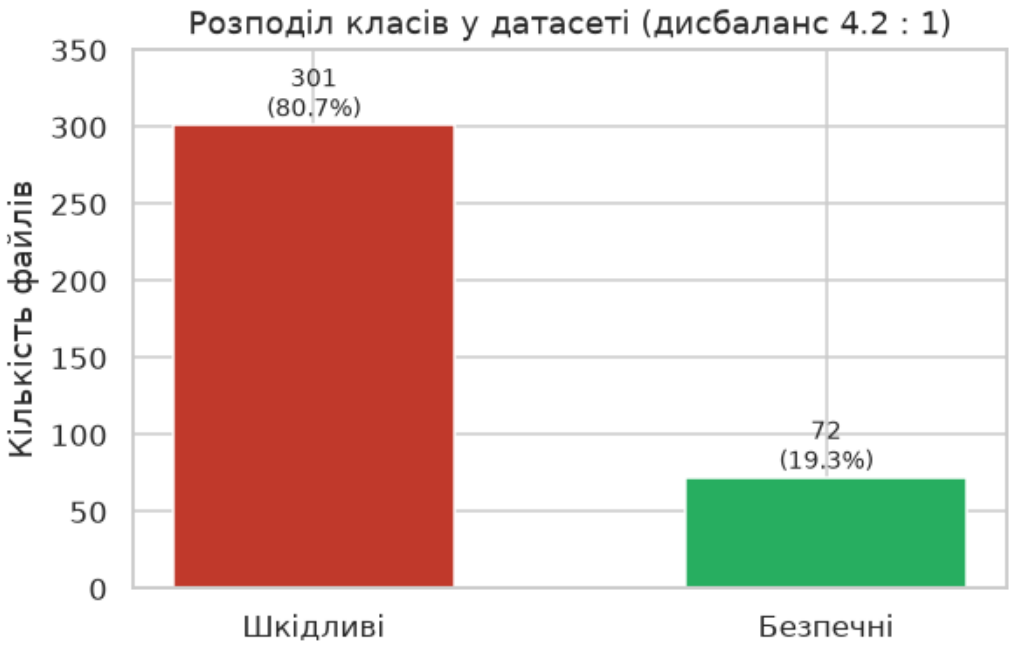


Рисунок 1: Розподіл класів у датасеті

3.2 Базова лінія

Перед навчанням будь-якої моделі визначено результат тривіального класифікатора, що завжди відповідає «шкідливий»:

Метрика	Значення
Точність	80.9 %
Recall щодо шкідливих	100 %
Recall щодо безпечних	**0 %**

Тобто **80.9 % точності досягається взагалі без навчання**. Це і є причина, чому далі точність не використовується як самостійна метрика — лише разом із precision, recall, F1 та матрицею невідповідностей.

3.3 Логістична регресія

Навчено три конфігурації моделі (`solver='liblinear'`, `max_iter=5000`):

1. базова з L2-регуляризацією, `C = 1`;
2. те саме з `class_weight='balanced'` — внесок рідкісного класу у функцію втрат збільшується обернено до його частоти;
3. модель на ознаках, відібраних L1-регуляризацією.

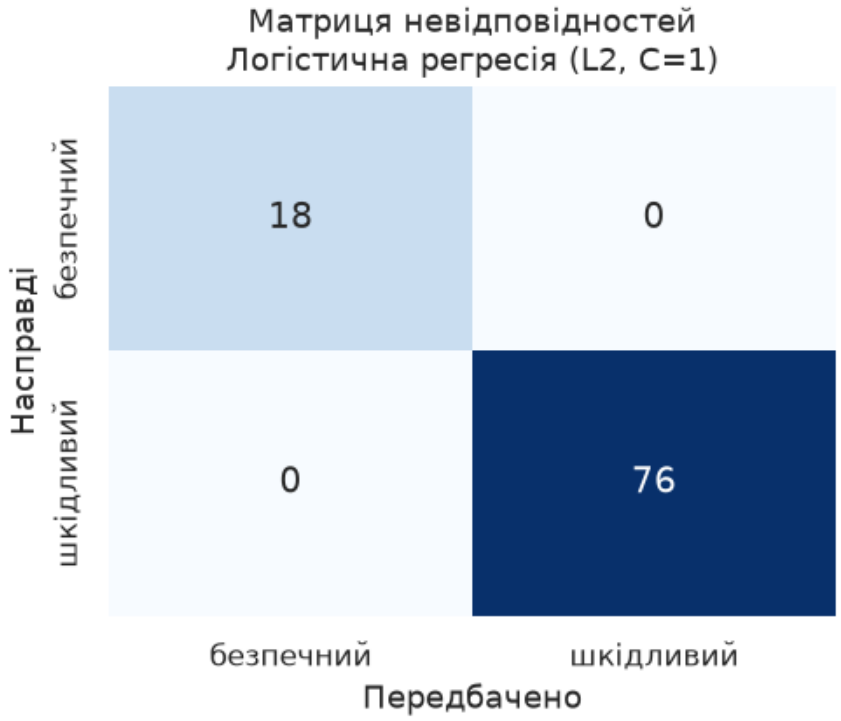


Рисунок 2: Матриця невідповідностей

3.4 Відбір ознак і виявлена пастка

L1-регуляризація (`C = 0.1`) залишила **лише 3 ненульові коефіцієнти з 503**.

Тут виявлено методичну пастку, варту окремої згадки. Стандартний спосіб `SelectFromModel(threshold="median")` у цьому випадку **не відбирає нічого**: оскільки L1 занулила більш ніж половину коефіцієнтів, медіана важливості дорівнює рівно нулю, і умова «важливість ≥ медіани» виконується для всіх ознак без винятку. Перша версія розрахунку через це «відібрала» всі 503 ознаки, і помилку було помічено лише тому, що виведена кількість збіглася з початковою. Коректний варіант — явний поріг `threshold=1e-10`, що залишає тільки дійсно ненульові ваги.

3.5 Оцінка складності задачі

Усі три моделі показали 100 % на тестовій вибірці. Ідеальний результат — привід для перевірки, а не для висновку про якість моделі. Виконано три перевірки:

- **витік даних**: знайдено 2 повні дублікати рядків, груп із суперечливими мітками — 0, тобто витоку між навчальною й тестовою вибірками немає;
- **перехресна перевірка** (стратифікована, 5 блоків) на всіх даних: F1 = 0.9967, тобто результат стійкий і не є наслідком вдалого поділу;
- **достатність однієї ознаки**: ознака `F_19` збігається з міткою у **98.4 %** випадків сама по собі.

Побудовано залежність якості від кількості відібраних ознак (`SelectKBest`, χ^2):

Кількість ознак	F1 (перехресна перевірка)
1	0.9902
2	0.9983
3	0.9983
10	0.9951
50	0.9967
503	0.9967

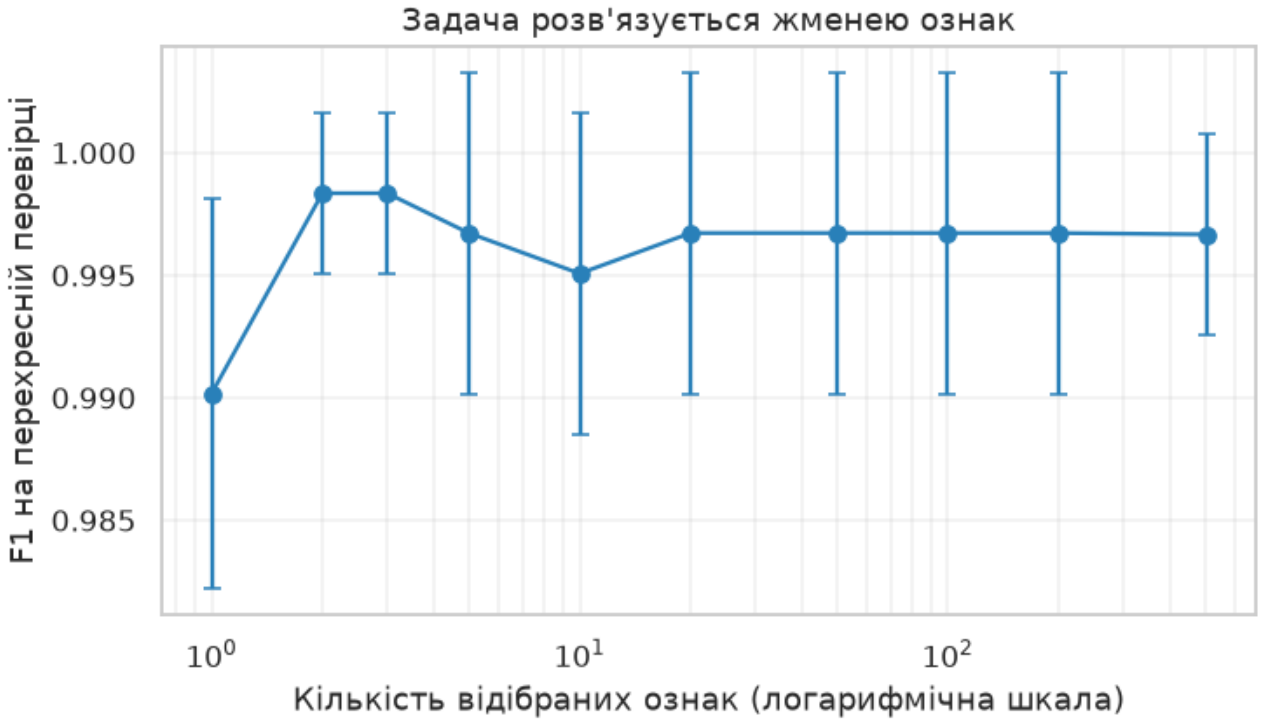


Рисунок 3: Залежність якості від кількості ознак

Двох ознак достатньо, щоб перевершити результат на всіх 503. Решта 501 ознака не додає інформації, а лише збільшує ризик перенавчання.

3.6 Інтерпретація коефіцієнтів

Згідно з методичкою, логістична регресія оцінює співвідношення шансів OR (odds ratio): для бінарної ознаки наявність ознаки множить шанси на клас «шкідливий» у `e^θ` разів.

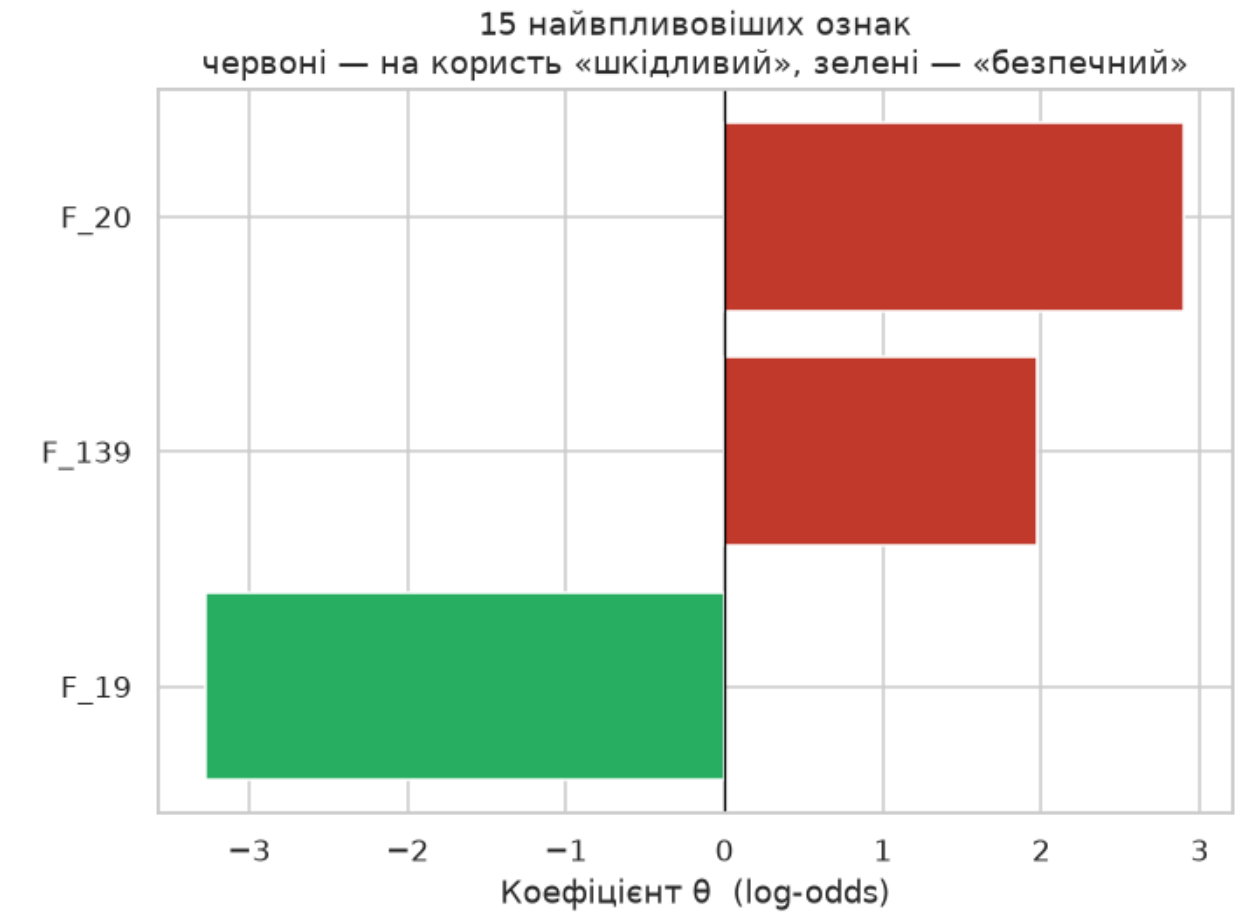


Рисунок 4: Найвпливовіші ознаки

3.7 Поріг відсікання та логістична функція

Поріг `k = 0.5` є довільним; його зміна торгує precision на recall.

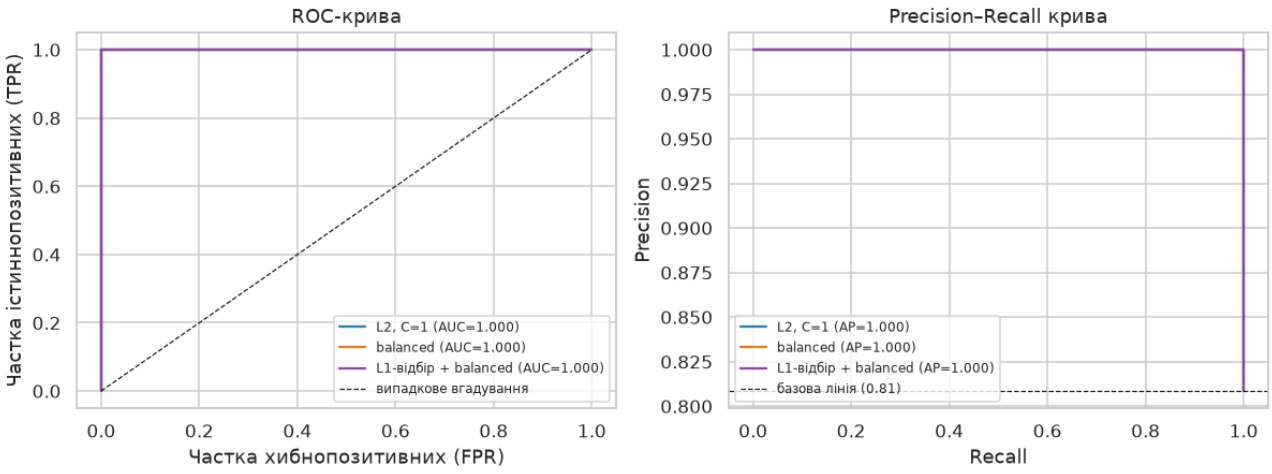


Рисунок 5: ROC та precision-recall криві

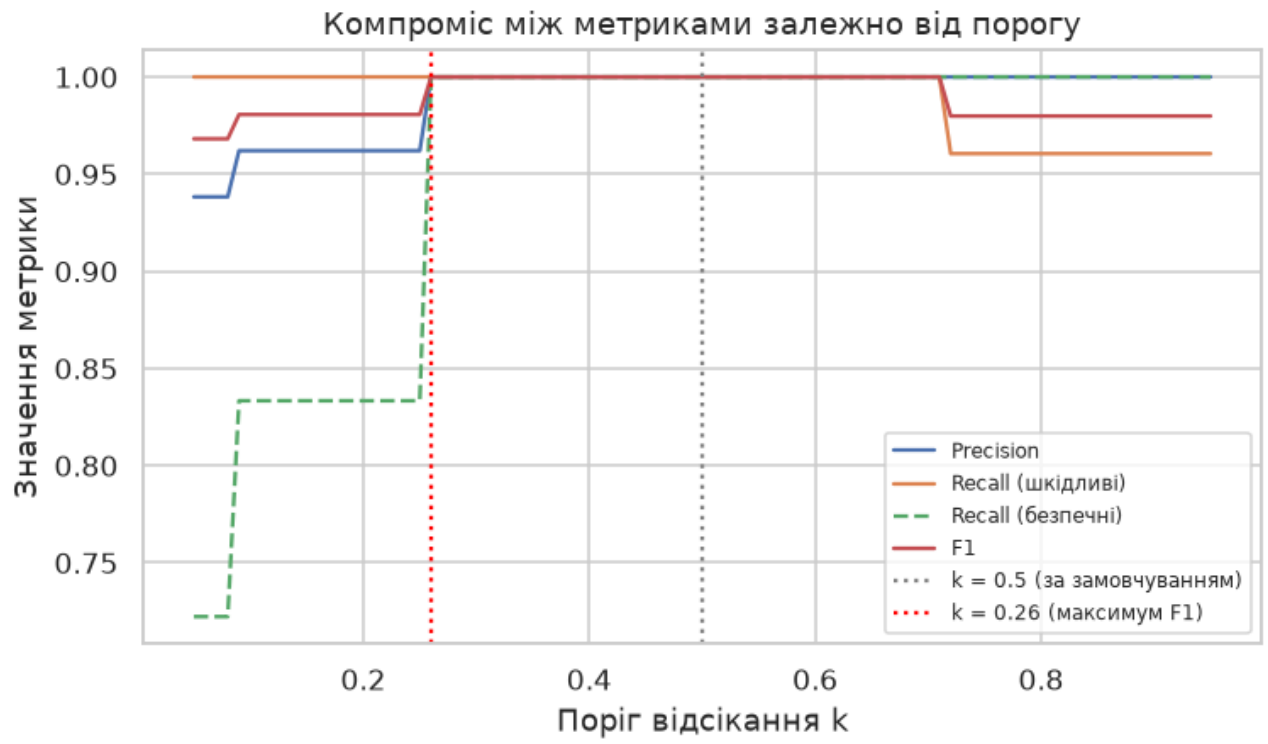


Рисунок 6: Вплив порогу відсікання

Розподіл передбачених ймовірностей практично вироджений — оцінки скупчені біля 0 та 1, проміжних значень майже немає. Це ще одне свідчення лінійної роздільності даних.

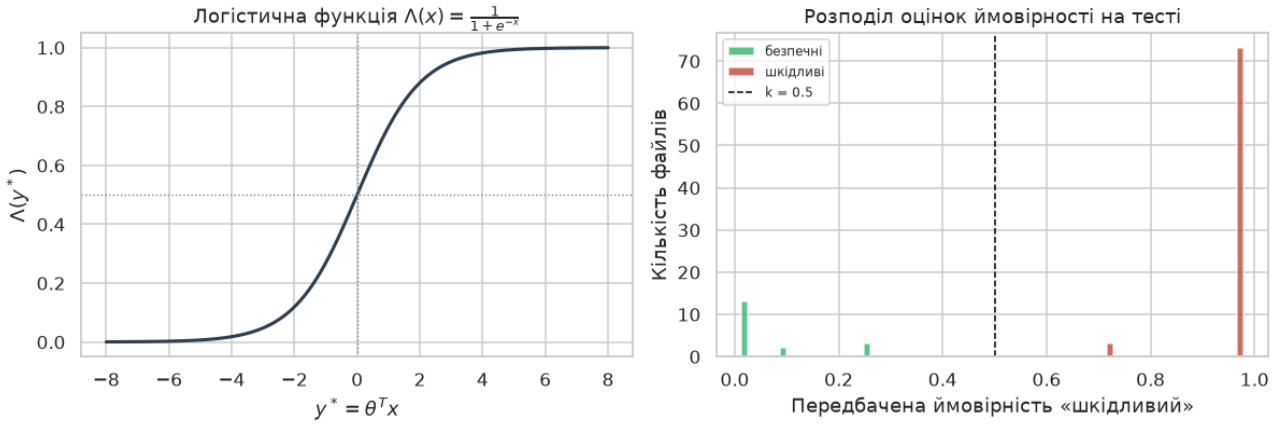


Рисунок 7: Логістична функція та розподіл оцінок

4 Результати

Модель	Точність	Precision	Recall (шкідл.)	Recall (безпеч.)	F1	ROC-AUC
Базова лінія («завжди шкідливий»)	0.809	0.809	1.000	0.000	0.894	0.500
Логістична регресія (L2, C=1)	1.000	1.000	1.000	1.000	1.000	1.000
Логістична регресія (balanced)	1.000	1.000	1.000	1.000	1.000	1.000
L1-відбір (3 ознаки) + balanced	1.000	1.000	1.000	1.000	1.000	1.000

Перехресна перевірка на всіх даних: **F1 = 0.9967 ± 0.0041**.

5 Аналіз

Модель працює, але задача виявилась тривіальною. Головний результат роботи — не 100 % точності, а з'ясування того, що цей результат майже нічого не говорить про логістичну регресію. Одна ознака дає F1 = 0.9902, дві — 0.9983. Ознаки в датасеті були сконструйовані авторами саме для відокремлення шкідливих файлів, тому лінійна роздільність закладена в них за побудовою.

Практичний наслідок: **цей датасет непридатний для порівняння алгоритмів**. Якщо на ньому запустити метод опорних векторів, дерево рішень чи kNN (практичні заняття 3, 2, 5), усі покажуть близько 100 %, і жодних змістовних висновків про переваги методів зробити не вдасться. Для тих робіт потрібні складніші дані.

Дисбаланс не зашкодив, але міг би. `class_weight='balanced'` не змінив результатів саме тому, що класи роздільні. На складніших даних дисбаланс 4.2 : 1 зсував би модель у бік більшості, а ціна помилки тут несиметрична: для антивіруса хибнопозитивне спрацювання (безпечна програма визнана шкідливою) зазвичай гірше за пропуск, бо користувач втрачає робочий застосунок.

Обмеження оцінки. Тестова вибірка містить лише 18 безпечних файлів. Помилка на двох із них зсунула б recall на 11 пунктів, тому одиничний поділ тут ненадійний — саме тому основною оцінкою є перехресна перевірка.

6 Висновки

1. Логістичну регресію реалізовано й застосовано до задачі класифікації виконуваних файлів; на тестовій вибірці досягнуто 100 %, на перехресній перевірці F1 = 0.9967.
2. Точність як самостійна метрика непридатна для незбалансованих даних: тривіальний класифікатор дає 80.9 %, не розпізнавши жодного безпечного файлу.
3. Проблему p > n (531 ознака проти 373 прикладів) вирішено L1-регуляризацією, яка звела модель до 3 ознак без втрати якості.
4. Виявлено, що задача розв’язується двома ознаками, тобто датасет лінійно роздільний за побудовою і не годиться для порівняння алгоритмів у наступних роботах.
5. Практичний урок: ідеальний результат вимагає перевірки на витік даних і тривіальність задачі, а типова реалізація відбору ознак через `threshold="median"` може мовчки не відбирати нічого.

7 Джерела

1. UCI Malware Executable Detection. URL: https://www.kaggle.com/datasets/piyushrumao/malware-executable-detection
2. Rumao P. Using Two Dimensional Hybrid Feature Dataset to Detect Malicious Executables // IJIRCCE. 2016. URL: http://www.ijircce.com/upload/2016/july/158_Using.pdf
3. Scikit-learn: Logistic Regression. URL: https://scikit-learn.org/stable/modules/linear_-model.html#logistic-regression